



L5A: Llist Processing

CS1101S: Programming Methodology

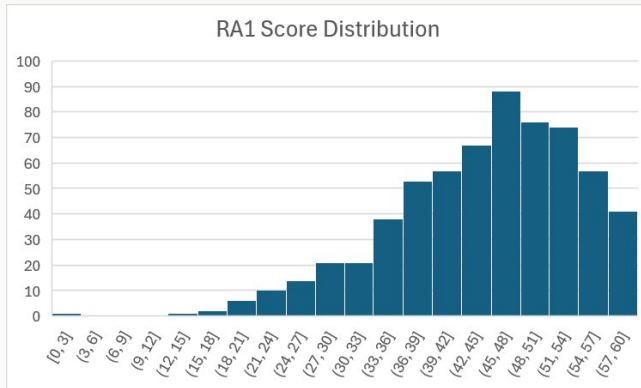
Sanka Rasnayaka

9th September, 2026

Version 1.0 — Built: 2026-09-08 18:08:27 SGT



Reading Assessment 1 — Marks Distribution





Some Remarks

- ▶ No “curve”! Your individual learning counts



Some Remarks

- ▶ No “curve”! Your individual learning counts
- ▶ It’s a marathon, not a sprint



Some Remarks

- ▶ No “curve”! Your individual learning counts
- ▶ It’s a marathon, not a sprint
- ▶ Make use of our resources



Some Remarks

- ▶ No “curve”! Your individual learning counts
- ▶ It’s a marathon, not a sprint
- ▶ Make use of our resources
- ▶ Learning attitude



Unit 1: Survey Reflection



Unit 1: Survey Reflection

► Academic Bottlenecks

- 1. Higher-order functions 51%
- 2. Lambda expressions 39%
- 3. Recursion 21%



Unit 1: Survey Reflection

- ▶ Academic Bottlenecks
 - 1. Higher-order functions 51%
 - 2. Lambda expressions 39%
 - 3. Recursion 21%
- ▶ Workload / Time Management / Assessments
 - 1. Quests! - "Optional but feels compulsory"
 - 2. Assessments



Unit 1: Survey Reflection

- ▶ Academic Bottlenecks
 - 1. Higher-order functions 51%
 - 2. Lambda expressions 39%
 - 3. Recursion 21%
- ▶ Workload / Time Management / Assessments
 - 1. Quests! - "Optional but feels compulsory"
 - 2. Assessments
- ▶ Noise level at lectures



Announcements

- ▶ Autograding has been fixed, check if your past submissions are graded, if any issues let us know!



Announcements

- ▶ Autograding has been fixed, check if your past submissions are graded, if any issues let us know!
- ▶ “Beautiful Runes” Contest Voting ...



Announcements

- ▶ Autograding has been fixed, check if your past submissions are graded, if any issues let us know!
- ▶ “Beautiful Runes” Contest Voting ...
- ▶ Contest “The Choreographer” Opening Soon



Outline

- ▶ Linked List Processing (2.2.1)
<https://sourceacademy.nus.edu.sg/sicpy/2.2.1>
- ▶ Higher-order Linked List Processing (2.2.2)
<https://sourceacademy.nus.edu.sg/sicpy/2.2.2>



Review: Pairs

```
p = pair(1, 2) # creates a pair  
head(p)      # accesses first component  
tail(p)      # accesses second component
```





Review: Linked List Discipline

- Definition: A **l**list is either **None** or a pair whose tail is a llist



Review: Linked List Discipline

- Definition: A **l**list is either **None** or a pair whose tail is a llist

```
pair(1, pair(2, pair(3, pair(4, None))))
```



Review: Linked List Discipline

- ▶ Definition: A **l**list is either **None** or a pair whose tail is a llist

```
pair(1, pair(2, pair(3, pair(4, None))))
```

- ▶ Displayed in box notation as, [1, [2, [3, [4, None]]]]



Review: Linked List Discipline

- ▶ Definition: A **llist** is either **None** or a pair whose tail is a llist

```
pair(1, pair(2, pair(3, pair(4, None))))
```

- ▶ Displayed in box notation as, [1, [2, [3, [4, None]]]]
- ▶ Displayed in llist notation as, `llist(1, 2, 3, 4)`



Review: Linked List Discipline

- Definition: A **llist** is either **None** or a pair whose tail is a llist

```
pair(1, pair(2, pair(3, pair(4, None))))
```

- Displayed in box notation as, [1, [2, [3, [4, None]]]]
- Displayed in llist notation as, **llist**(1, 2, 3, 4)
- Displayed in box-and-pointer as,





Refinement of Linked List Discipline

► Definition:

A llist of a certain data type is `None` or a pair whose head is of that data type and whose tail is a llist of that data type



Refinement of Linked List Discipline

- ▶ Definition:
A llist of a certain data type is `None` or a pair whose head is of that data type and whose tail is a llist of that data type
- ▶ Examples:



Refinement of Linked List Discipline

► Definition:

A llist of a certain data type is `None` or a pair whose head is of that data type and whose tail is a llist of that data type

► Examples:

```
# a llist of numbers  
l1istA = pair(1, pair(2, None))
```





Refinement of Linked List Discipline

► Definition:

A llist of a certain data type is `None` or a pair whose head is of that data type and whose tail is a llist of that data type

► Examples:

```
# a llist of numbers
llistA = pair(1, pair(2, None))

# a llist of booleans
llistB = llist(True, True, False, True)
```



Refinement of Linked List Discipline

► Definition:

A llist of a certain data type is `None` or a pair whose head is of that data type and whose tail is a llist of that data type

► Examples:

```
# a llist of numbers
llistA = pair(1, pair(2, None))

# a llist of booleans
llistB = llist(True, True, False, True)

# a llist of strings
llistC = llist("X", "Y", "Z")
```



Refinement of Linked List Discipline

- ▶ Definition:

A llist of a certain data type is `None` or a pair whose head is of that data type and whose tail is a llist of that data type

- ▶ Examples:

```
# a llist of numbers
l1istA = pair(1, pair(2, None))

# a llist of booleans
l1istB = llist(True, True, False, True)

# a llist of strings
l1istC = llist("X", "Y", "Z")

l1istD = None # can be a llist of any type
```



Review: Computing the Length of a Linked List

► Specification:



Review: Computing the Length of a Linked List

- Specification:
The length of the empty llist is 0, and the length of a non-empty llist is one more than the length of its tail



Review: Computing the Length of a Linked List

- ▶ Specification:
The length of the empty llist is 0, and the length of a non-empty llist is one more than the length of its tail
- ▶ Implementation (recursive version):



Review: Computing the Length of a Linked List

- Specification:
The length of the empty llist is 0, and the length of a non-empty llist is one more than the length of its tail
- Implementation (recursive version):

```
def length(xs):  
    return 0  
        if is_none(xs)  
        else 1 + length(tail(xs))
```





Review: Computing the Length of a Linked List

- ▶ Iterative version:



Review: Computing the Length of a Linked List

► Iterative version:

```
def length_iter(xs):  
    def helper(ys, counted_so_far):  
        return (counted_so_far  
                if is_none(ys)  
                else helper(tail(ys), counted_so_far + 1))  
    return helper(xs, 0)
```



Appending Two Lists

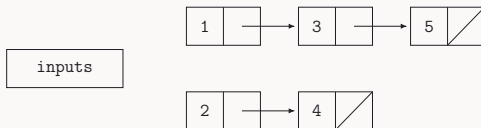
- ▶ Wanted: Append `l1ist(1, 3, 5)` and `l1ist(2, 4)` to produce result equal to `l1ist(1, 3, 5, 2, 4)`
- ▶ `append(l1ist(1, 3, 5), l1ist(2, 4))`

Appending Two Lists

- ▶ Wanted: Append `l1ist(1, 3, 5)` and `l1ist(2, 4)` to produce result equal to `l1ist(1, 3, 5, 2, 4)`
- ▶ `append(l1ist(1, 3, 5), l1ist(2, 4))`
 $\Rightarrow$ `l1ist(1, 3, 5, 2, 4)`

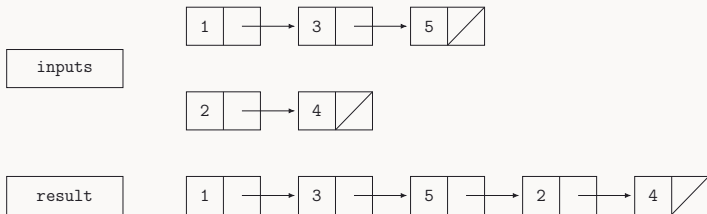
Appending Two Lists

- ▶ Wanted: Append `l1list(1, 3, 5)` and `l1list(2, 4)` to produce result equal to `l1list(1, 3, 5, 2, 4)`
- ▶ `append(l1list(1, 3, 5), l1list(2, 4))`
 $\Rightarrow$ `l1list(1, 3, 5, 2, 4)`



Appending Two Lists

- ▶ Wanted: Append `l1list(1, 3, 5)` and `l1list(2, 4)` to produce result equal to `l1list(1, 3, 5, 2, 4)`
- ▶ `append(l1list(1, 3, 5), l1list(2, 4))`
 $\Rightarrow$ `l1list(1, 3, 5, 2, 4)`





Appending Two Lists

- First attempt:

```
append1 = pair
```



Appending Two Lists

- ▶ First attempt:

```
append1 = pair
```

- ▶ Does it work?

```
append1(l1list(1, 3, 5), l1list(2, 4))
```



Appending Two Lists

- ▶ First attempt:

```
append1 = pair
```

- ▶ Does it work?

```
append1(llist(1, 3, 5), llist(2, 4))
```

```
⇒ pair(llist(1, 3, 5), llist(2, 4))
```



Appending Two Lists

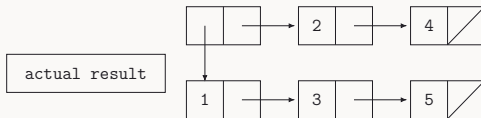
- First attempt:

```
append1 = pair
```

- Does it work?

```
append1(llist(1, 3, 5), llist(2, 4))
```

```
⇒ pair(llist(1, 3, 5), llist(2, 4))
```



Appending Two Lists

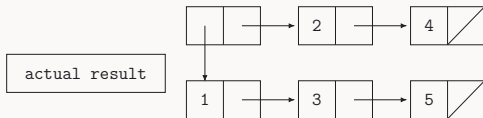
- ▶ First attempt:

```
append1 = pair
```

- ▶ Does it work?

```
append1(llist(1, 3, 5), llist(2, 4))
```

```
⇒ pair(llist(1, 3, 5), llist(2, 4))
```





Strategy for Append

- ▶ `append(l1list1, l1list2):`
 - If `l1list1` is empty, return `l1list2`



Strategy for Append

- ▶ `append(l1, l2):`
 - If `l1` is empty, return `l2`
 - Otherwise, wishful thinking!



Strategy for Append

- ▶ `append(l1, l2):`
 - If `l1` is empty, return `l2`
 - Otherwise, wishful thinking!
 - Append the tail of `l1` to `l2`



Strategy for Append

- ▶ `append(llist1, llist2):`
 - If `llist1` is empty, return `llist2`
 - Otherwise, wishful thinking!
 - Append the tail of `llist1` to `llist2`
 - Form a pair of the head of `llist1` and the result



Implementation of Append in Python

- ▶ Second attempt:



Implementation of Append in Python

► Second attempt:

```
def append2(xs, ys):  
    return (ys  
            if is_none(xs)  
            else pair(head(xs), append2(tail(xs), ys)))
```



Implementation of Append in Python

- ▶ Second attempt:

```
def append2(xs, ys):  
    return (ys  
            if is_none(xs)  
            else pair(head(xs), append2(tail(xs), ys)))
```

- ▶ Order of growth in time?



Implementation of Append in Python

- ▶ Second attempt:

```
def append2(xs, ys):  
    return (ys  
            if is_none(xs)  
            else pair(head(xs), append2(tail(xs), ys)))
```

- ▶ Order of growth in time?
- ▶ Order of growth in space?

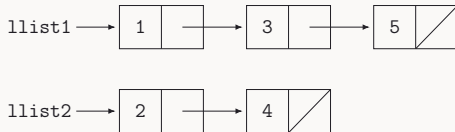
Append: Example Run

```
def append2(xs, ys):  
    return (ys  
            if is_none(xs)  
            else pair(head(xs), append2(tail(xs), ys)))  
  
l1list1 = l1list(1, 3, 5)  
l1list2 = l1list(2, 4)  
result = append2(l1list1, l1list2)
```



Append: Example Run

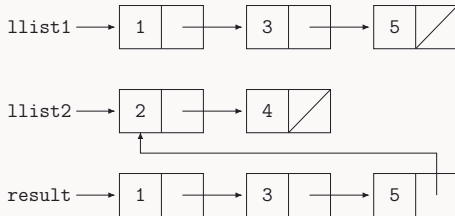
```
def append2(xs, ys):  
    return (ys  
            if is_none(xs)  
            else pair(head(xs), append2(tail(xs), ys)))  
  
l1list1 = llist(1, 3, 5)  
l1list2 = llist(2, 4)  
result = append2(l1list1, l1list2)
```





Append: Example Run

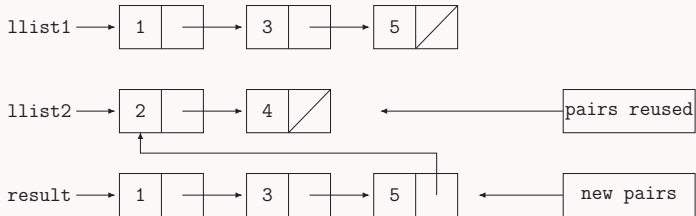
```
def append2(xs, ys):  
    return (ys  
            if is_none(xs)  
            else pair(head(xs), append2(tail(xs), ys)))  
  
l1list1 = llist(1, 3, 5)  
l1list2 = llist(2, 4)  
result = append2(l1list1, l1list2)
```





Append: Example Run

```
def append2(xs, ys):  
    return (ys  
            if is_none(xs)  
            else pair(head(xs), append2(tail(xs), ys)))  
  
l1list1 = l1list(1, 3, 5)  
l1list2 = l1list(2, 4)  
result = append2(l1list1, l1list2)
```



λ

Refining Space Complexity



Refining Space Complexity

- ▶ In Unit 1, space complexity was about **deferred operations**.



Refining Space Complexity

- ▶ In Unit 1, space complexity was about **deferred operations**.
- ▶ In Unit 2, programs can also create **compound data values** such as pairs.



Refining Space Complexity

- ▶ In Unit 1, space complexity was about **deferred operations**.
- ▶ In Unit 2, programs can also create **compound data values** such as pairs.
- ▶ Need to consider the space needed for `pair` objects.

Refining Space Complexity

- ▶ In Unit 1, space complexity was about **deferred operations**.
- ▶ In Unit 2, programs can also create **compound data values** such as pairs.
- ▶ Need to consider the space needed for `pair` objects.

```
p = pair(1, 2) # creates one new pair  
  
head(p)  
tail(p)
```



Refining Space Complexity

- ▶ In Unit 1, space complexity was about **deferred operations**.
- ▶ In Unit 2, programs can also create **compound data values** such as pairs.
- ▶ Need to consider the space needed for `pair` objects.

```
p = pair(1, 2) # creates one new pair  
  
head(p)  
tail(p)
```

- ▶ `head(p)` and `tail(p)` do not create new pairs.



Refining Space Complexity

- ▶ In Unit 1, space complexity was about **deferred operations**.
- ▶ In Unit 2, programs can also create **compound data values** such as pairs.
- ▶ Need to consider the space needed for `pair` objects.

```
p = pair(1, 2) # creates one new pair  
  
head(p)  
tail(p)
```



- ▶ `head(p)` and `tail(p)` do not create new pairs.
- ▶ Referring to or passing `p` does not copy the pair.

Refining Space Complexity

- ▶ In Unit 1, space complexity was about **deferred operations**.
- ▶ In Unit 2, programs can also create **compound data values** such as pairs.
- ▶ Need to consider the space needed for `pair` objects.

```
p = pair(1, 2) # creates one new pair  
  
head(p)  
tail(p)
```

- ▶ `head(p)` and `tail(p)` do not create new pairs.
- ▶ Referring to or passing `p` does not copy the pair.

Only a call to `pair(...)` creates a new pair.



Which Pairs Contribute to Space?

```
def f(x):  
    local_pair = pair("some head", "some tail")  
    return 0 if x == 0 else f(x - 1)
```

```
f(100)
```



Which Pairs Contribute to Space?

```
def f(x):  
    local_pair = pair("some head", "some tail")  
    return 0 if x == 0 else f(x - 1)
```

```
f(100)
```

- ▶ `pair(...)` is called $O(n)$ times.



Which Pairs Contribute to Space?

```
def f(x):  
    local_pair = pair("some head", "some tail")  
    return 0 if x == 0 else f(x - 1)
```

```
f(100)
```

- ▶ `pair(...)` is called $O(n)$ times.
- ▶ `local_pair` is no longer needed after the next recursive call.

Which Pairs Contribute to Space?

```
def f(x):  
    local_pair = pair("some head", "some tail")  
    return 0 if x == 0 else f(x - 1)
```

```
f(100)
```

- ▶ `pair(...)` is called $O(n)$ times.
- ▶ `local_pair` is no longer needed after the next recursive call.
- ▶ At any point, at most **one pair** needs to be remembered.



Which Pairs Contribute to Space?

```
def f(x):  
    local_pair = pair("some head", "some tail")  
    return 0 if x == 0 else f(x - 1)
```

```
f(100)
```

- ▶ `pair(...)` is called $O(n)$ times.
- ▶ `local_pair` is no longer needed after the next recursive call.
- ▶ At any point, at most **one pair** needs to be remembered.

Refinement

We do not count the total number of pairs created, instead count the **maximum number of pairs that need to be remembered at any point**.



Can We Count Pairs in the Stepper?

```
def f(x, p):  
    return None if x == 0 else f(x - 1, pair(p, p))  
  
f(10, None)
```





Can We Count Pairs in the Stepper?

```
def f(x, p):  
    return None if x == 0 else f(x - 1, pair(p, p))  
  
f(10, None)
```

- Each recursive call creates exactly **one new pair**.

Can We Count Pairs in the Stepper?

```
def f(x, p):  
    return None if x == 0 else f(x - 1, pair(p, p))  
  
f(10, None)
```

- ▶ Each recursive call creates exactly **one new pair**.
- ▶ After n recursive calls, there are $O(n)$ pairs to remember.



Can We Count Pairs in the Stepper?

```
def f(x, p):  
    return None if x == 0 else f(x - 1, pair(p, p))  
  
f(10, None)
```



- ▶ Each recursive call creates exactly **one new pair**.
- ▶ After n recursive calls, there are $O(n)$ pairs to remember.
- ▶ Both components of the new pair refer to the **same existing pair**.



Can We Count Pairs in the Stepper?

```
def f(x, p):  
    return None if x == 0 else f(x - 1, pair(p, p))  
  
f(10, None)
```

- ▶ Each recursive call creates exactly **one new pair**.
- ▶ After n recursive calls, there are $O(n)$ pairs to remember.
- ▶ Both components of the new pair refer to the **same existing pair**.
- ▶ How does Stepper handle this?



Limitation of Stepper in Counting Pairs

```
p1 = pair(None, None)
p2 = pair(p1, p1)
p3 = pair(p2, p2)
```



Limitation of Stepper in Counting Pairs

```
p1 = pair(None, None)
p2 = pair(p1, p1)
p3 = pair(p2, p2)
```



- ▶ The Stepper gives us a **textual representation** of pairs.



Limitation of Stepper in Counting Pairs

```
p1 = pair(None, None)
p2 = pair(p1, p1)
p3 = pair(p2, p2)
```



- ▶ The Stepper gives us a **textual representation** of pairs.
- ▶ The same pair may therefore appear multiple times.



Limitation of Stepper in Counting Pairs

```
p1 = pair(None, None)
p2 = pair(p1, p1)
p3 = pair(p2, p2)
```



- ▶ The Stepper gives us a **textual representation** of pairs.
- ▶ The same pair may therefore appear multiple times.
- ▶ Counting [,] in the Stepper does not tell us how many distinct pairs exist.
- ▶ We need a better model for values, sharing, and what needs to be remembered.



Limitation of Stepper in Counting Pairs

```
p1 = pair(None, None)
p2 = pair(p1, p1)
p3 = pair(p2, p2)
```



- ▶ The Stepper gives us a **textual representation** of pairs.
- ▶ The same pair may therefore appear multiple times.
- ▶ Counting [,] in the Stepper does not tell us how many distinct pairs exist.
- ▶ We need a better model for values, sharing, and what needs to be remembered.

→ **Unit 3! :)**



Reversing a Llist

- ▶ First attempt:



Reversing a Llist

► First attempt:

```
def reverse1(lst):
```





Reversing a Llist

► First attempt:

```
def reverse1(lst):  
    return (None  
            if is_none(lst)
```





Reversing a Llist

► First attempt:

```
def reverse1(lst):  
    return (None  
            if is_none(lst)  
            else pair(reverse1(tail(lst)), head(lst)))
```





Reversing a Llist

► First attempt:

```
def reverse1(lst):  
    return (None  
            if is_none(lst)  
            else pair(reverse1(tail(lst)), head(lst)))
```

► Does it work?



Reversing a Llist

► First attempt:

```
def reverse1(lst):  
    return (None  
            if is_none(lst)  
            else pair(reverse1(tail(lst)), head(lst)))
```

► Does it work?
`reverse1(llist(1, 2, 3))`



Reversing a Llist

► First attempt:

```
def reverse1(lst):  
    return (None  
            if is_none(lst)  
            else pair(reverse1(tail(lst)), head(lst)))
```

► Does it work?

`reverse1(llist(1, 2, 3))`

$\Rightarrow$ `pair(pair(pair(None, 3), 2), 1)`

Reversing a Llist

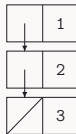
- First attempt:

```
def reverse1(lst):  
    return (None  
            if is_none(lst)  
            else pair(reverse1(tail(lst)), head(lst)))
```

- Does it work?

`reverse1(llist(1, 2, 3))`

$\Rightarrow$ `pair(pair(pair(None, 3), 2), 1)`





Reversing a Llist

- ▶ Second attempt:



Reversing a Llist

► Second attempt:

```
def reverse2(lst):  
    return (None  
            if is_none(lst)  
            else append(reverse2(tail(lst)),  
                          llist(head(lst))))
```



Reversing a Llist

- ▶ Second attempt:

```
def reverse2(lst):  
    return (None  
            if is_none(lst)  
            else append(reverse2(tail(lst)),  
                          llist(head(lst))))
```

- ▶ Does it work?
reverse2(llist(1, 2, 3))



Reversing a Llist

- ▶ Second attempt:

```
def reverse2(lst):  
    return (None  
            if is_none(lst)  
            else append(reverse2(tail(lst)),  
                          llist(head(lst))))
```

- ▶ Does it work?
reverse2(llist(1, 2, 3))
⇒ llist(3, 2, 1)
- ▶ Correct! But what about its run time?



Reversing a Llist

- ▶ Third attempt:



Reversing a Llist

► Third attempt:

```
def reverse3(xs):
```





Reversing a Llist

► Third attempt:

```
def reverse3(xs):  
    def rev(original, reversed):
```





Reversing a Llist

► Third attempt:

```
def reverse3(xs):  
    def rev(original, reversed):  
  
  
    return rev(xs, None)
```



Reversing a Llist

- ▶ Third attempt:

```
def reverse3(xs):  
    def rev(original, reversed):  
        return (reversed  
                if is_none(original)  
                else rev(rev(original), reversed))  
  
    return rev(xs, None)
```

- ▶ Order of growth in time?



Reversing a Llist

- ▶ Third attempt:

```
def reverse3(xs):  
    def rev(original, reversed):  
        return (reversed  
                if is_none(original)  
                else rev(tail(original),  
                        pair(head(original), reversed)))  
    return rev(xs, None)
```

- ▶ Order of growth in time?



Reversing a Llist

- ▶ Third attempt:

```
def reverse3(xs):  
    def rev(original, reversed):  
        return (reversed  
                if is_none(original)  
                else rev(tail(original),  
                        pair(head(original), reversed)))  
    return rev(xs, None)
```

- ▶ Order of growth in time?



Reversing a Llist

- ▶ Third attempt:

```
def reverse3(xs):  
    def rev(original, reversed):  
        return (reversed  
                if is_none(original)  
                else rev(tail(original),  
                        pair(head(original), reversed)))  
    return rev(xs, None)
```

- ▶ Order of growth in time?
- ▶ Order of growth in space?



Outline

- ▶ Linked List Processing (2.2.1)
<https://sourceacademy.nus.edu.sg/sicpy/2.2.1>
- ▶ Higher-order Linked List Processing (2.2.2)
<https://sourceacademy.nus.edu.sg/sicpy/2.2.2>



Example: Scaling a Linked List

- ▶ Let us scale all elements of a llist by a factor k:



Example: Scaling a Linked List

- ▶ Let us scale all elements of a llist by a factor k:

```
scale_llist(llist(1, 2, 3), 10)
```



Example: Scaling a Linked List

- ▶ Let us scale all elements of a llist by a factor k:

```
scale_llist(llist(1, 2, 3), 10)  
⇒ llist(10, 20, 30)
```



Example: Scaling a Linked List

- ▶ Let us scale all elements of a llist by a factor k:

```
def scale_llist(xs, k):
```



```
scale_llist(llist(1, 2, 3), 10)
```

```
⇒ llist(10, 20, 30)
```



Example: Scaling a Linked List

- ▶ Let us scale all elements of a llist by a factor k:

```
def scale_llist(xs, k):  
    return (None  
            if is_none(xs)
```



```
scale_llist(llist(1, 2, 3), 10)  
⇒ llist(10, 20, 30)
```

Example: Scaling a Linked List

- ▶ Let us scale all elements of a llist by a factor k:

```
def scale_llist(xs, k):  
    return (None  
            if is_none(xs)  
            else pair(k * head(xs),  
                      scale_llist(tail(xs), k)))
```

scale_llist(llist(1, 2, 3), 10)

⇒ llist(10, 20, 30)



Example: Squaring a Linked List

- Let us square all elements of a llist:

```
def square_llist(xs):  
    square = lambda x: x * x  
    return (None  
            if is_none(xs)  
            else pair(square(head(xs)),  
                      square_llist(tail(xs))))
```



Example: Squaring a Linked List

- Let us square all elements of a llist:

```
def square_llist(xs):  
    square = lambda x: x * x  
    return (None  
            if is_none(xs)  
            else pair(square(head(xs)),  
                      square_llist(tail(xs))))
```

`square_llist(llist(1, 2, 3, 4))`

$\Rightarrow$ `llist(1, 4, 9, 16)`



Abstraction: map

- **Mapping** means applying a given function f **element-wise** to a given llist xs



Abstraction: map

- ▶ **Mapping** means applying a given function f **element-wise** to a given llist xs
- ▶ The result is a llist consisting of the results of applying f to each element of xs

Abstraction: map

- ▶ **Mapping** means applying a given function f **element-wise** to a given llist xs
- ▶ The result is a llist consisting of the results of applying f to each element of xs

```
def map(f, xs):  
    return (None  
            if is_none(xs)  
            else pair(f(head(xs)), map(f, tail(xs))))
```





Abstraction: map

```
def map(f, xs):  
    return (None  
            if is_none(xs)  
            else pair(f(head(xs)), map(f, tail(xs))))
```





Rewriting using `map` Abstraction

```
def map(f, xs):  
    return (None  
            if is_none(xs)  
            else pair(f(head(xs)), map(f, tail(xs))))
```



Rewriting using `map` Abstraction

```
def map(f, xs):  
    return (None  
            if is_none(xs)  
            else pair(f(head(xs)), map(f, tail(xs))))  
  
def scale_llist(xs, k):
```



Rewriting using map Abstraction

```
def map(f, xs):  
    return (None  
            if is_none(xs)  
            else pair(f(head(xs)), map(f, tail(xs))))  
  
def scale_llist(xs, k):  
    return map(lambda x: k * x, xs)
```



Rewriting using `map` Abstraction

```
def map(f, xs):  
    return (None  
            if is_none(xs)  
            else pair(f(head(xs)), map(f, tail(xs))))  
  
def scale_llist(xs, k):  
    return map(lambda x: k * x, xs)  
  
def square_llist(xs):
```



Rewriting using map Abstraction

```
def map(f, xs):  
    return (None  
            if is_none(xs)  
            else pair(f(head(xs)), map(f, tail(xs))))  
  
def scale_llist(xs, k):  
    return map(lambda x: k * x, xs)  
  
def square_llist(xs):  
    return map(lambda x: x * x, xs)
```



Making a Copy of a Llist

```
def copy(xs):  
    return map(lambda x: x, xs)
```

```
original = llist(1, 3, 5)  
my_copy = copy(original)
```





Making a Copy of a Llist

```
def copy(xs):  
    return map(lambda x: x, xs)
```

```
original = llist(1, 3, 5)  
my_copy = copy(original)
```





Making a Copy of a Llist

```
def copy(xs):  
    return map(lambda x: x, xs)
```

```
original = llist(1, 3, 5)  
my_copy = copy(original)
```





Check-In time!

Don't get lost! Let's `map` our way to the check-in!

Look for Check-In I5A in the Source Academy!



Example: Taking Even Numbers from a Llist

- ▶ Take only even numbers from a llist:



Example: Taking Even Numbers from a Llist

- ▶ Take only even numbers from a llist:

```
even_numbers(llist(1, 2, 3, 4, 5, 6))
```



Example: Taking Even Numbers from a Llist

- Take only even numbers from a llist:

```
even_numbers(llist(1, 2, 3, 4, 5, 6))  
⇒ llist(2, 4, 6)
```



Example: Taking Even Numbers from a Llist

- Take only even numbers from a llist:

```
def even_numbers(xs):
```



```
even_numbers(llist(1, 2, 3, 4, 5, 6))  
⇒ llist(2, 4, 6)
```



Example: Taking Even Numbers from a Llist

- Take only even numbers from a llist:

```
def even_numbers(xs):  
    return (None  
            if is_none(xs)
```



```
even_numbers(llist(1, 2, 3, 4, 5, 6))  
⇒ llist(2, 4, 6)
```



Example: Taking Even Numbers from a Llist

- Take only even numbers from a llist:

```
def even_numbers(xs):  
    return (None  
            if is_none(xs)  
            else pair(head(xs), even_numbers(tail(xs)))  
                if head(xs) % 2 == 0
```

```
even_numbers(llist(1, 2, 3, 4, 5, 6))  
⇒ llist(2, 4, 6)
```



Example: Taking Even Numbers from a Llist

- Take only even numbers from a llist:

```
def even_numbers(xs):  
    return (None  
            if is_none(xs)  
            else pair(head(xs), even_numbers(tail(xs)))  
                if head(xs) % 2 == 0  
                else even_numbers(tail(xs)))
```

```
even_numbers(llist(1, 2, 3, 4, 5, 6))  
⇒ llist(2, 4, 6)
```



Abstraction: filter



Abstraction: filter

```
def filter(pred, xs):  
    return (None  
            if is_none(xs)  
            else pair(head(xs), filter(pred, tail(xs)))  
                if pred(head(xs))  
                else filter(pred, tail(xs)))
```



Rewriting even_numbers using filter abstraction:

```
def filter(pred, xs):  
    return (None  
            if is_none(xs)  
            else pair(head(xs), filter(pred, tail(xs)))  
                if pred(head(xs))  
                else filter(pred, tail(xs)))  
  
def even_numbers(xs):  
    return filter(lambda x: x % 2 == 0, xs)
```



Example: Summing Elements of a Llist

- ▶ Compute the sum of all elements of a llist of numbers:



Example: Summing Elements of a Llist

- Compute the sum of all elements of a llist of numbers:

```
def llist_sum(xs):
```





Example: Summing Elements of a Llist

- Compute the sum of all elements of a llist of numbers:

```
def llist_sum(xs):  
    return (0  
            if is_none(xs)
```





Example: Summing Elements of a Llist

- Compute the sum of all elements of a llist of numbers:

```
def llist_sum(xs):  
    return (0  
            if is_none(xs)  
            else head(xs) + llist_sum(tail(xs)))
```





Example: Summing Elements of a Llist

- Compute the sum of all elements of a llist of numbers:

```
def llist_sum(xs):  
    return (0  
            if is_none(xs)  
            else head(xs) + llist_sum(tail(xs)))
```





Example: Summing Elements of a Llist

- Compute the sum of all elements of a llist of numbers:

```
def llist_sum(xs):  
    return (0  
            if is_none(xs)  
            else head(xs) + llist_sum(tail(xs)))
```

- `llist_sum(llist(1, 2, 3))`

Example: Summing Elements of a Llist

- ▶ Compute the sum of all elements of a llist of numbers:

```
def llist_sum(xs):  
    return (0  
            if is_none(xs)  
            else head(xs) + llist_sum(tail(xs)))
```

- ▶ `llist_sum(llist(1, 2, 3))`
- ▶ $\Rightarrow 1 + \text{llist_sum}(\text{llist}(2, 3))$



Example: Summing Elements of a Llist

- ▶ Compute the sum of all elements of a llist of numbers:

```
def llist_sum(xs):  
    return (0  
            if is_none(xs)  
            else head(xs) + llist_sum(tail(xs)))
```

- ▶ `llist_sum(llist(1, 2, 3))`
- ▶ $\Rightarrow 1 + \text{llist_sum}(\text{llist}(2, 3))$
- ▶ $\Rightarrow 1 + (2 + \text{llist_sum}(\text{llist}(3)))$



Example: Summing Elements of a Llist

- ▶ Compute the sum of all elements of a llist of numbers:

```
def llist_sum(xs):  
    return (0  
            if is_none(xs)  
            else head(xs) + llist_sum(tail(xs)))
```

- ▶ `llist_sum(llist(1, 2, 3))`
- ▶ $\Rightarrow 1 + \text{llist_sum}(\text{llist}(2, 3))$
- ▶ $\Rightarrow 1 + (2 + \text{llist_sum}(\text{llist}(3)))$
- ▶ $\Rightarrow 1 + (2 + (3 + \text{llist_sum}(\text{None})))$



Example: Summing Elements of a Llist

- ▶ Compute the sum of all elements of a llist of numbers:

```
def llist_sum(xs):  
    return (0  
            if is_none(xs)  
            else head(xs) + llist_sum(tail(xs)))
```

- ▶ `llist_sum(llist(1, 2, 3))`
- ▶ $\Rightarrow 1 + \text{llist_sum}(\text{llist}(2, 3))$
- ▶ $\Rightarrow 1 + (2 + \text{llist_sum}(\text{llist}(3)))$
- ▶ $\Rightarrow 1 + (2 + (3 + \text{llist_sum}(\text{None})))$
- ▶ $\Rightarrow 1 + (2 + (3 + 0))$



Example: Summing Elements of a Llist

- ▶ Compute the sum of all elements of a llist of numbers:

```
def llist_sum(xs):  
    return (0  
            if is_none(xs)  
            else head(xs) + llist_sum(tail(xs)))
```

- ▶ `llist_sum(llist(1, 2, 3))`
- ▶ $\Rightarrow 1 + \text{llist_sum}(\text{llist}(2, 3))$
- ▶ $\Rightarrow 1 + (2 + \text{llist_sum}(\text{llist}(3)))$
- ▶ $\Rightarrow 1 + (2 + (3 + \text{llist_sum}(\text{None})))$
- ▶ $\Rightarrow 1 + (2 + (3 + 0))$
- ▶ $\Rightarrow 6$ right-to-left folding



Abstraction: reduce



Abstraction: reduce

```
def reduce(op, initial, xs):  
    return (initial  
            if is_none(xs)  
            else op(head(xs),  
                    reduce(op, initial, tail(xs))))
```





Abstraction: reduce

```
def reduce(op, initial, xs):  
    return (initial  
            if is_none(xs)  
            else op(head(xs),  
                    reduce(op, initial, tail(xs))))
```

- Rewriting `l1ist_sum` using `reduce` abstraction:



Abstraction: reduce

```
def reduce(op, initial, xs):  
    return (initial  
            if is_none(xs)  
            else op(head(xs),  
                    reduce(op, initial, tail(xs))))
```

- Rewriting `l1ist_sum` using `reduce` abstraction:

```
def l1ist_sum(xs):
```

Abstraction: reduce

```
def reduce(op, initial, xs):  
    return (initial  
            if is_none(xs)  
            else op(head(xs),  
                    reduce(op, initial, tail(xs))))
```

- Rewriting `l1ist_sum` using `reduce` abstraction:

```
def l1ist_sum(xs):  
    return reduce(lambda x, y: x + y, 0, xs)
```



Abstraction: reduce

```
def reduce(op, initial, xs):  
    return (initial  
            if is_none(xs)  
            else op(head(xs),  
                    reduce(op, initial, tail(xs))))
```



λ

The Trio



The Trio

```
def map(f, xs):  
    return (None  
            if is_none(xs)  
            else pair(f(head(xs)), map(f, tail(xs))))  
  
def filter(pred, xs):  
    return (None  
            if is_none(xs)  
            else pair(head(xs), filter(pred, tail(xs)))  
                  if pred(head(xs))  
                  else filter(pred, tail(xs)))  
  
def reduce(op, initial, xs):  
    return (initial  
            if is_none(xs)  
            else op(head(xs),  
                    reduce(op, initial, tail(xs))))
```



Summary

- ▶ Llist processing:
 - length, append, reverse
- ▶ Higher-order llist processing
 - map, filter, reduce
- ▶ Refinement of space complexity
 - Count the maximum number of pairs that need to be remembered at any point.